

ToolMap

Application de gestion d'inventaire
pour garage automobile

Documentation Technique Complète

Version 1.0 — Avril 2026

43 fichiers source • 6 modèles de données • 12 endpoints API

Docker Compose • Next.js 15 (App Router) • TypeScript • MySQL 8 • Prisma ORM • Tailwind CSS • Recharts

Table des matières

1. Présentation générale

1.1 Objectif de l'application

1.2 Stack technique

1.3 Système de coordonnées

2. Architecture technique

2.1 Vue d'ensemble

2.2 Arborescence du projet

2.3 Configuration Docker

3. Installation et déploiement

3.1 Prérequis

3.2 Installation pas à pas

3.3 Variables d'environnement

4. Modèle de données

4.1 Diagramme entité-relation

4.2 Détail des tables

4.3 Catégories par défaut

5. API REST

5.1 Catégories

5.2 Items

5.3 Emplacements & Grilles

5.4 Mouvements & Export CSV

5.5 Prêts

5.6 Dashboard

5.7 Sauvegarde & Restauration

6. Pages de l'application

6.1 Navigation (Sidebar)

6.2 Accueil (Dashboard)

6.3 Outils / Matériels / Consommables

6.4 Paramétrage

7. Plan 2D des casiers

7.1 Génération de grille

7.2 Indicateurs visuels

7.3 Interaction avec un casier

7.4 Modification et suppression

8. Système d'historique

8.1 Génération automatique

8.2 Consultation et filtrage

8.3 Export CSV

9. Gestion des prêts

10. Sauvegarde et restauration

11. Thème et design

12. Composants React

13. Annexes

A. Commandes utiles

B. Cycle de vie d'un Item

C. Flux de données

D. Format des photos

E. Sécurité

F. Dépannage

1. Présentation générale

1.1 Objectif de l'application

ToolMap est une application web complète de gestion d'inventaire conçue pour un garage automobile. Elle répond aux besoins suivants :

- **Inventaire complet** : gérer des *Outils*, *Matériels* et *Consommables* avec photos, catégories et descriptions
- **Localisation physique** : situer chaque objet via un système de coordonnées 4 niveaux (Lieu → Emplacement → Ligne → Colonne)
- **Plan 2D interactif** : visualiser l'organisation des casiers sur une grille CSS Grid
- **Traçabilité totale** : historique automatique de chaque mouvement (ajout, modification, suppression, prêt, retour, vente, épuisement)
- **Gestion des prêts** : suivi des clients, dates prévues, retards
- **Tableau de bord** : statistiques et graphiques camembert (Top 10)
- **Export CSV** : extraction des historiques au format tableur
- **Sauvegarde / Restauration** : backup et restore complet de la base de données en JSON

1.2 Stack technique

Composant	Technologie	Version	Rôle
Conteneurisation	Docker / Docker Compose	20+	Orchestration des services
Frontend & Backend	Next.js (App Router)	15.x	SSR, API Routes, rendu React
Langage	TypeScript	5.x	Typage statique
Base de données	MySQL	8.0	Stockage persistant
ORM	Prisma	6.x	Modélisation et requêtes type-safe
CSS	Tailwind CSS	3.4	Styles utilitaires
Graphiques	Recharts	2.x	Camemberts sur le Dashboard
HTTPS	Node.js https natif	—	Serveur HTTPS sur port 443

1.3 Système de coordonnées

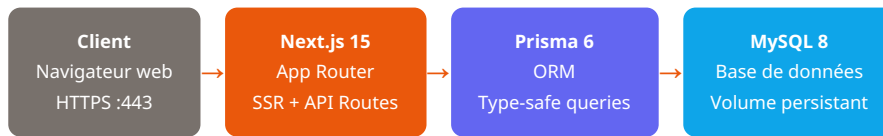
Chaque emplacement physique est identifié par **4 niveaux hiérarchiques** :

Niveau	Format	Exemples	Description
Lieu	Texte prédéfini	Grange, Grange_etage, Garage, Cabane_jardin	Bâtiment ou zone
Emplacement	E + numéro	E0, E1, E11, E999	Meuble / étagère dans le lieu
Ligne	Lettre A-Z	A, B, C, ... Z	Rangée verticale (max 26)
Colonne	Chiffre 1-N	1, 2, 3, ... 50	Position horizontale

Exemple complet : Garage - E11-B4 = Bâtiment Garage, meuble E11, rangée B, case 4.

2. Architecture technique

2.1 Vue d'ensemble



Le flux de données suit un modèle **client** → **HTTPS** → **Next.js API** → **Prisma** → **MySQL**. Le serveur HTTPS est un reverse proxy Node.js qui transmet les requêtes au serveur standalone Next.js (port 3000 interne).

2.2 Arborescence du projet

```
toolmap/
├─ docker-compose.yml      # Orchestration Docker (2 services)
├─ Dockerfile              # Build multi-stage Next.js
├─ server.js               # Wrapper HTTPS reverse proxy
├─ generate-ssl.sh         # Script génération certificats
├─ package.json            # Dépendances NPM
├─ .env                    # Variables d'environnement
├─ prisma/
│  └─ schema.prisma        # Schéma BDD (6 modèles)
│  └─ seed.ts              # 30 catégories par défaut
├─ ssl/                    # Certificats SSL (générés)
├─ docs/
│  └─ documentation.html   # Cette documentation
└─ src/
   └─ lib/
      └─ prisma.ts          # Singleton Prisma Client
   └─ components/          # 9 composants React
      ├── Sidebar.tsx       # Navigation latérale
      ├── Modal.tsx         # Fenêtre modale générique
      ├── ImageZoom.tsx     # Miniature cliquable + zoom
      ├── ItemForm.tsx      # Formulaire CRUD item
      ├── ItemListPage.tsx  # Liste filtrée + recherche
      ├── ItemDetailModal.tsx # Fiche détail complète
      ├── MovementLog.tsx   # Historique + export CSV
      ├── CategoriesManager.tsx # CRUD catégories
      ├── GridPlan.tsx      # Plan 2D interactif
      └─ BackupRestore.tsx  # Sauvegarde/restauration BDD
   └─ app/
      ├── layout.tsx        # Layout principal + sidebar
      ├── globals.css       # Styles Tailwind personnalisés
      ├── page.tsx          # Dashboard (Accueil)
      ├── outils/page.tsx   # Page Outils
      ├── materiels/page.tsx # Page Matériels
      ├── consommables/page.tsx # Page Consommables
      ├── parametrage/page.tsx # Page Paramétrage (3 onglets)
      └─ api/               # 12 endpoints API REST
         ├── categories/    # GET, POST, PUT, DELETE
         ├── items/         # GET, POST, PUT, DELETE
         ├── locations/     # GET, POST
         ├── grid/          # GET, POST, PUT, DELETE
         ├── movements/     # GET + export CSV
         ├── loans/         # GET, POST, PUT
         ├── dashboard/     # GET (stats + charts)
         └─ backup/         # GET (export) + POST (import)
```

2.3 Configuration Docker

docker-compose.yml — 2 services

Service	Image	Port exposé	Volume	Dépendance
db	mysql:8.0	3306	mysql_data (persistant)	—
web	Build local (Dockerfile)	443 (HTTPS)	./ssl (read-only)	db (healthcheck)

Dockerfile — Build multi-stage

Stage	Base	Actions
base	node:20-alpine	Install dépendances, copie schema Prisma, <code>npm install</code> , <code>prisma generate</code> , <code>next build</code>
runner	node:20-alpine	Copie du build standalone, renommage <code>server.js</code> → <code>server.standalone.js</code> , ajout du wrapper HTTPS

Serveur HTTPS (server.js)

Le serveur de production fonctionne en deux couches :

1. **Next.js standalone** (`server.standalone.js`) : écoute en HTTP sur le port 3000 (interne)
2. **Reverse proxy HTTPS** (`server.js`) : écoute sur le port 443 avec les certificats SSL, redirige vers le port 3000

3. Installation et déploiement

3.1 Prérequis

Logiciel	Version minimale	Vérification
Docker	20.0+	<code>docker --version</code>
Docker Compose	v2+	<code>docker compose version</code>
OpenSSL	1.1+	<code>openssl version</code>

3.2 Installation pas à pas

1 Générer les certificats SSL

```
cd toolmap
chmod +x generate-ssl.sh
./generate-ssl.sh
```

Crée `ssl/key.pem` et `ssl/cert.pem` (certificat auto-signé, valide 365 jours).

2 Construire et lancer les conteneurs

```
docker compose up -d --build
```

Première exécution : ~2 minutes (téléchargement images + build).

3 Initialiser la base de données

```
# Appliquer le schéma
docker exec toolmap-web ./node_modules/.bin/prisma db push

# Insérer les catégories par défaut
docker exec toolmap-web ./node_modules/.bin/tsx prisma/seed.ts
```

4 Accéder à l'application

```
https://localhost
```

Note : Le navigateur affichera un avertissement car le certificat est auto-signé. Cliquez « Avancé » → « Continuer vers localhost ».

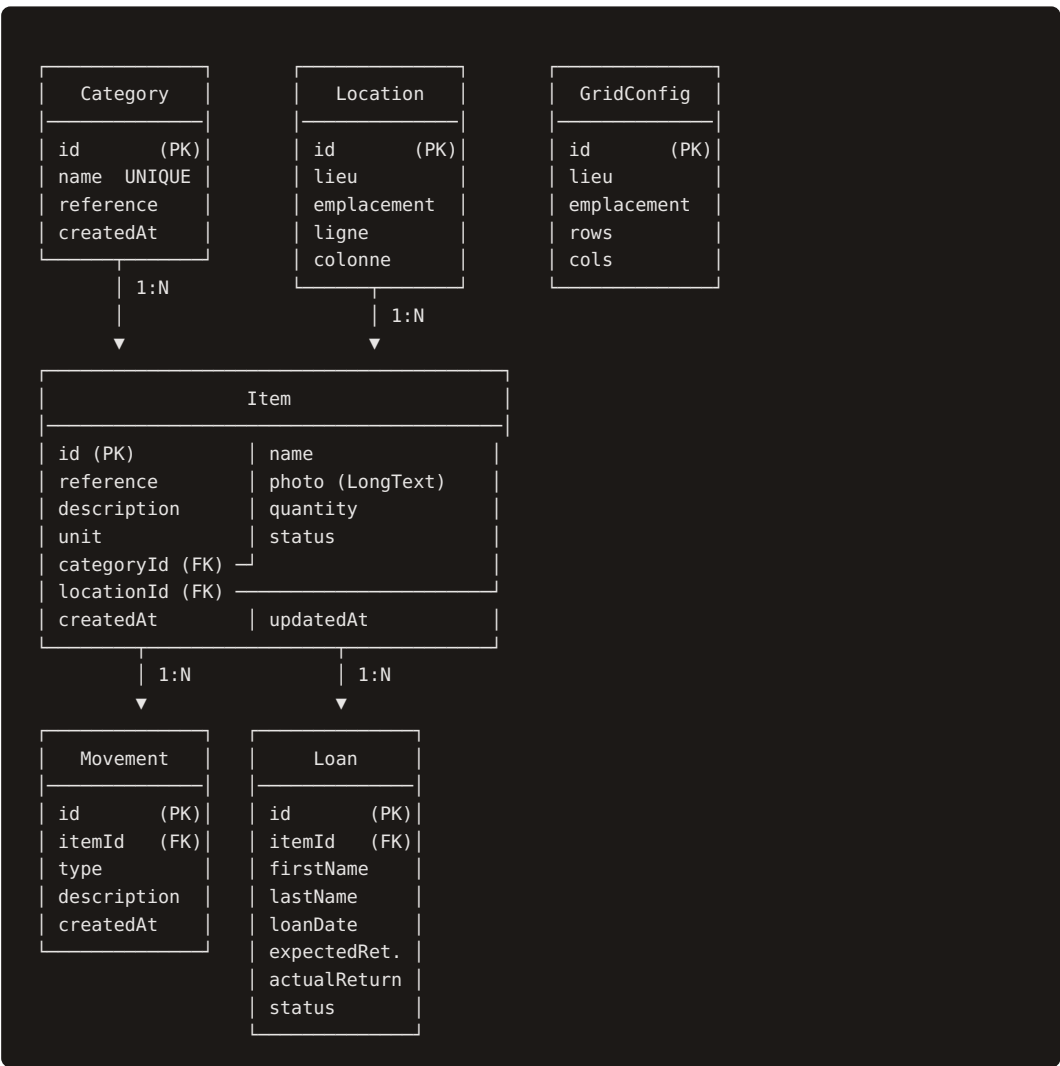
3.3 Variables d'environnement

Variable	Service	Valeur par défaut
<code>DATABASE_URL</code>	web	<code>mysql://toolmap:toolmap_pass_2024@db:3306/toolmap</code>
<code>MYSQL_ROOT_PASSWORD</code>	db	<code>toolmap_root_2024</code>
<code>MYSQL_DATABASE</code>	db	<code>toolmap</code>
<code>MYSQL_USER</code>	db	<code>toolmap</code>
<code>MYSQL_PASSWORD</code>	db	<code>toolmap_pass_2024</code>

Sécurité production : Modifiez impérativement tous les mots de passe par défaut et utilisez un vrai certificat SSL (Let's Encrypt).

4. Modèle de données

4.1 Diagramme entité-relation



4.2 Détail des tables

Table Category

Champ	Type MySQL	Contraintes	Description
id	INT AUTO_INCREMENT	PRIMARY KEY	Identifiant unique
name	VARCHAR(100)	UNIQUE	Nom de la catégorie
reference	VARCHAR(50)	INDEX	Outils, Matériels, ou Consommables
createdAt	DATETIME	DEFAULT NOW()	Date de création

Table Item

Champ	Type MySQL	Contraintes	Description
id	INT AUTO_INCREMENT	PRIMARY KEY	Identifiant unique

name	VARCHAR(200)	NOT NULL	Nom de l'objet
reference	VARCHAR(50)	INDEX	Type : Outils / Matériels / Consommables
photo	LONGTEXT	NULLABLE	Image encodée en Base64 (data URI)
description	TEXT	NULLABLE	Description libre
quantity	INT	DEFAULT 1	Quantité en stock
unit	VARCHAR(20)	DEFAULT 'unitaire'	unitaire, litre, metre
categoryId	INT	FK → Category, INDEX	Catégorie de l'item
locationId	INT	FK → Location, NULLABLE, INDEX	Emplacement physique (null = non assigné)
status	VARCHAR(20)	DEFAULT 'actif', INDEX	actif, vendu, vide
createdAt	DATETIME	DEFAULT NOW()	Date de création
updatedAt	DATETIME	AUTO UPDATE	Dernière modification

Table Movement

Champ	Type MySQL	Contraintes	Description
id	INT AUTO_INCREMENT	PRIMARY KEY	Identifiant unique
itemId	INT	FK → Item (CASCADE), INDEX	Item concerné
type	VARCHAR(30)	INDEX	Ajout, Modif, Suppression, Pret, Rendu, Vendu, Vide
description	TEXT	NULLABLE	Détail du mouvement
createdAt	DATETIME	DEFAULT NOW(), INDEX	Date du mouvement

Table Loan

Champ	Type MySQL	Contraintes	Description
id	INT AUTO_INCREMENT	PRIMARY KEY	Identifiant unique
itemId	INT	FK → Item (CASCADE), INDEX	Item prêté
firstName	VARCHAR(100)	NOT NULL	Prénom du client
lastName	VARCHAR(100)	NOT NULL	Nom du client
loanDate	DATETIME	DEFAULT NOW()	Date du prêt
expectedReturn	DATETIME	NOT NULL	Date de retour prévue
actualReturn	DATETIME	NULLABLE	Date de retour effectif
status	VARCHAR(20)	DEFAULT 'en_cours', INDEX	en_cours, rendu

Table Location

Champ	Type MySQL	Contraintes	Description
id	INT AUTO_INCREMENT	PRIMARY KEY	Identifiant unique
lieu	VARCHAR(50)	INDEX	Grange, Grange_etage, Garage, Cabane_jardin

emplacement	VARCHAR(10)	—	E0 à E999
ligne	VARCHAR(2)	—	A à Z
colonne	INT	—	1 à N

Contrainte unique : (lieu, emplacement, ligne, colonne)

Table GridConfig

Champ	Type MySQL	Contraintes	Description
id	INT AUTO_INCREMENT	PRIMARY KEY	Identifiant unique
lieu	VARCHAR(50)	—	Lieu du plan
emplacement	VARCHAR(10)	—	Identifiant emplacement
rows	INT	—	Nombre de lignes générées
cols	INT	—	Nombre de colonnes générées

Contrainte unique : (lieu, emplacement)

4.3 Catégories par défaut (seed)

Outils (10)	Matériels (10)	Consommables (10)
Clés	Levage	Huiles
Tournevis	Soudure	Filtres
Pinces	Diagnostic	Visserie
Marteaux	Pneumatique	Joints
Scies	Éclairage	Adhésifs
Mesure	Nettoyage	Abrasifs
Douilles	Rangement	Lubrifiants
Limes	Protection	Peinture
Extracteurs	Électrique	Électricité
Électroportatif	Hydraulique	Nettoyants

5. API REST

Toutes les routes sont sous `/api/`. Le format d'échange est **JSON** sauf pour l'export CSV.

5.1 Catégories `/api/categories`

Méthode	Route	Paramètres	Description
GET	<code>/api/categories</code>	?reference=Outils	Lister (avec compteur items)
POST	<code>/api/categories</code>	Body: {name, reference}	Créer une catégorie
PUT	<code>/api/categories/:id</code>	Body: {name, reference}	Modifier
DELETE	<code>/api/categories/:id</code>	—	Supprimer

5.2 Items `/api/items`

Méthode	Route	Paramètres	Description
GET	<code>/api/items</code>	? reference=&categoryId=&search=&status=&locationId=	Lister (dédupliqué par ID)
POST	<code>/api/items</code>	Body complet	Créer + mouvement « Ajout » auto
GET	<code>/api/items/:id</code>	—	Détail avec historique et prêts
PUT	<code>/api/items/:id</code>	Body partiel	Modifier + mouvement « Modif » auto
DELETE	<code>/api/items/:id</code>	—	Supprimer + mouvement « Suppression » auto

Paramètre status : actif (défaut) = items actifs ; hidden = items vendu/vidé (historique masqué).

5.3 Emplacements & Grilles

Méthode	Route	Description
GET	<code>/api/locations?lieu=Garage</code>	Lister les emplacements (avec compteur items)
POST	<code>/api/locations</code>	Créer un emplacement unitaire
GET	<code>/api/grid?lieu=Garage</code>	Lister les configurations de grille
POST	<code>/api/grid</code>	Générer une grille (upsert config + création locations)
PUT	<code>/api/grid/:id</code>	Modifier dimensions (détache items hors grille)
DELETE	<code>/api/grid/:id</code>	Supprimer plan + détacher tous les items

Comportement à la suppression/réduction : Les items dans les casiers supprimés sont automatiquement détachés (`locationId = null`). Aucun item n'est perdu.

5.4 Mouvements & Export CSV

Méthode	Route	Description
GET	<code>/api/movements?type=&search=&hidden=true</code>	Consulter l'historique (500 derniers max)
GET	<code>/api/movements/export?reference=Outils&hidden=false</code>	Télécharger le CSV (UTF-8 + BOM)

Format du fichier CSV exporté

```
Date,Article,Reference,Type,Description,Statut
08/04/2026 14:32:00,"Clé à molette",Outils,Ajout,"Ajout de ""Clé à molette"" (Outils)",actif
08/04/2026 15:10:00,"Huile 5W40",Consommables,Vide,"Statut changé vers ""vide""",vide
```

5.5 Prêts `/api/loans`

Méthode	Route	Description
GET	<code>/api/loans?status=en_cours</code>	Lister les prêts (avec détail item)
POST	<code>/api/loans</code>	Créer un prêt + mouvement « Pret » auto
PUT	<code>/api/loans/:id</code>	Retour de prêt + mouvement « Rendu » auto

5.6 Dashboard `/api/dashboard`

Méthode	Route	Réponse
GET	<code>/api/dashboard</code>	<code>{topOutils[], topConsommables[], activeLoans[], stats{}}</code>

5.7 Sauvegarde & Restauration `/api/backup`

Méthode	Route	Description
GET	<code>/api/backup</code>	Export JSON complet (toutes les tables)
POST	<code>/api/backup</code>	Import JSON (supprime puis restaure toutes les données)

Structure du fichier de sauvegarde

```
{
  "version": "1.0",
  "date": "2026-04-08T12:00:00.000Z",
  "data": {
    "categories": [...],
    "locations": [...],
    "gridConfigs": [...],
    "items": [...],
    "movements": [...],
    "loans": [...]
  }
}
```

Attention : La restauration **remplace intégralement** toutes les données. Effectuez toujours une sauvegarde avant de restaurer.

6. Pages de l'application

6.1 Navigation (Sidebar)

Barre latérale fixe (256px) avec 5 liens, présente sur toutes les pages :

Icône	Label	Route	Description
	Accueil	/	Tableau de bord avec statistiques
	Outils	/outils	Inventaire outils
	Matériels	/materiels	Inventaire matériels
	Consommables	/consommables	Inventaire consommables
	Paramétrage	/parametrage	Plans, catégories, backup

6.2 Accueil (Dashboard)

La page d'accueil affiche :

- **4 cartes statistiques** : Total items, Outils, Matériels, Consommables
- **2 graphiques camembert** (Recharts PieChart) côte à côte :
 - Top 10 outils les plus utilisés (par nombre de mouvements)
 - Top 10 consommables les plus utilisés
- **Tableau des prêts en cours** : Client, Article, Type, Date, Retour prévu (rouge si en retard)

6.3 Outils / Matériels / Consommables

Ces 3 pages partagent le même composant `ItemListPage` paramétré par référence :

Fonctionnalités de la liste

- **Barre de recherche** : filtre en temps réel sur nom, catégorie, référence
- **Filtre par catégorie** : menu déroulant avec compteur d'items
- **Grille responsive** : 1 à 4 colonnes selon la largeur
- **Miniatures cliquables** : clic pour zoom plein écran
- **Déduplication** : par ID automatique
- **Tri** : par catégorie (A-Z) puis par nom (A-Z)

Fiche détail (modal)

Au clic sur un item, un modal large s'ouvre avec :

- Photo zoomable, tags (référence, catégorie, statut coloré)
- Quantité, unité, emplacement
- **Boutons d'action contextuels** :
 - **Prêter** : ouvre un formulaire de prêt (tous types)
 - **Vendu** : change le statut (Matériels uniquement)
 - **Vide** : change le statut (Consommables uniquement)
- Liste des prêts en cours avec bouton « Rendu »
- Historique complet des mouvements (coloré par type)

Historique des mouvements

Sous la grille, deux boutons :

- **Historique des mouvements** : tous les mouvements filtrables + **Export CSV**
- **Historique masqué (Vendu/Vide)** : mouvements d'items vendus ou vides + **Export CSV**

6.4 Paramétrage

Page à 3 onglets :

Onglet	Composant	Fonctionnalités
Plan 2D des casiers	GridPlan	Générer, visualiser, modifier, supprimer des grilles
Catégories	CategoriesManager	CRUD complet, filtre par référence
Sauvegarde / Restauration	BackupRestore	Export JSON (backup) et import JSON (restore)

7. Plan 2D des casiers

7.1 Génération de grille

Processus de création d'un plan :

1. Sélectionner le **Lieu** (menu déroulant : Grange, Grange_etage, Garage, Cabane_jardin)
2. Saisir l'**Emplacement** (E0, E1, E11...)
3. Définir le **nombre de lignes** (1 à 26 → A à Z)
4. Définir le **nombre de colonnes** (1 à 50)
5. Cliquer « **Générer le plan** »

L'API crée automatiquement une `GridConfig` et toutes les `Location` correspondantes via upsert.

7.2 Indicateurs visuels

	1
	2
	3
	4
	A
E0-A1	occupé
E0-A2	
E0-A3	
E0-A4	occupé
B	
E0-B1	
E0-B2	occupé
E0-B3	
E0-B4	
C	
E0-C1	
E0-C2	
E0-C3	
E0-C4	occupé

État	Fond	Bordure	Texte
Casier vide	Gris clair transparent	Pointillée grise	Gris clair (garage-200)

Casier occupé	Bleu foncé semi-transparent	Solide bleue	Blanc bleu (blue-100) + label « occupé »
---------------	-----------------------------	--------------	--

7.3 Interaction avec un casier

Au clic sur une case, un **modal** s'ouvre avec :

- **Titre** : nom complet du casier (ex: « Garage — E0-B4 »)
- **Liste des items** présents : miniature photo, nom, catégorie, quantité, unité
- **Bouton « + Ajouter un élément »** : ouvre un formulaire `ItemForm` avec l'emplacement pré-rempli

7.4 Modification et suppression

Chaque plan affiché dispose de deux boutons en haut à droite :

Bouton « Modifier »

- Ouvre un modal avec les dimensions actuelles (lignes / colonnes)
- Le lieu et l'emplacement sont affichés en lecture seule
- Si la grille est **réduite** : les items dans les casiers supprimés sont **détachés** (emplacement remis à vide)
- Si la grille est **agrandie** : de nouvelles locations sont créées

Bouton « Supprimer »

- Demande confirmation
- Détache tous les items de la grille (`locationId = null`)
- Supprime toutes les locations et la configuration du plan
- **Aucun item n'est perdu** : ils passent simplement en « non assigné »

8. Système d'historique

8.1 Génération automatique

Chaque changement d'état génère **automatiquement** une entrée dans la table `Movement` :

Action utilisateur	Type mouvement	Description générée automatiquement
Création d'item	Ajout	« Ajout de "Nom" (Réf) »
Modification de champs	Modif	Détail des champs modifiés (Nom: X → Y, Quantité: 5 → 3)
Suppression d'item	Suppression	« Suppression de "Nom" »
Création de prêt	Pret	« Prêt à Prénom Nom - retour prévu le JJ/MM/AAAA »
Retour de prêt	Rendu	« Retour de prêt par Prénom Nom »
Marquage « Vendu »	Vendu	« Statut changé vers "vendu" »
Marquage « Vide »	Vide	« Statut changé vers "vide" »

8.2 Consultation et filtrage

Deux journaux accessibles depuis chaque page (Outils, Matériels, Consommables) via des boutons sous la liste :

- **Historique des mouvements** : tous les mouvements, filtrable par recherche texte libre
- **Historique masqué (Vendu/Vide)** : uniquement les mouvements de type Vendu et Vide

Chaque entrée affiche : date, nom de l'article, badge coloré du type, description.

8.3 Export CSV

Chaque modal d'historique dispose d'un bouton « **Exporter CSV** » qui :

- Télécharge un fichier `.csv` encodé en **UTF-8 avec BOM** (compatible Excel)
- Colonnes : Date, Article, Reference, Type, Description, Statut
- Filtre automatiquement par la référence de l'onglet courant (Outils, Matériels, Consommables)
- Nom du fichier : `historique_Outils_mouvements.csv` ou `historique_Outils_masque.csv`

9. Gestion des prêts

Création d'un prêt

1. Ouvrir la fiche détail d'un item actif
2. Cliquer « **Prêter** »
3. Remplir : Prénom, Nom, Date de retour prévue
4. Valider → mouvement `Pret` généré automatiquement

Retour d'un prêt

1. Ouvrir la fiche détail de l'item
2. Dans « Prêts en cours », cliquer « **Rendu** »
3. Statut → `rendu` , date de retour effectif enregistrée, mouvement `Rendu` créé

Suivi sur le Dashboard

Tableau central listant tous les prêts actifs. Les retours en retard sont signalés en **rouge avec** .

10. Sauvegarde et restauration

Accessible dans **Paramétrage** → onglet « **Sauvegarde / Restauration** ».

Sauvegarde (Export)

- Bouton « **Télécharger la sauvegarde** »
- Exporte l'intégralité de la base (6 tables) dans un fichier JSON
- Nom : `toolmap_backup_2026-04-08.json`
- Inclut : categories, items (avec photos Base64), locations, gridConfigs, movements, loans

Restauration (Import)

- Bouton « **Restaurer depuis un fichier** » (rouge, indique le danger)
- Demande de **confirmation** avant exécution
- Processus :
 1. Suppression de toutes les données existantes (dans l'ordre des dépendances)
 2. Insertion des données du fichier (dans l'ordre des dépendances)
 3. Conservation des IDs originaux
- Message de succès ou d'erreur affiché

Précaution : Toujours effectuer une sauvegarde avant toute restauration. L'opération est irréversible.

11. Thème et design

11.1 Palette de couleurs « Garage-friendly »

Nom Tailwind	Hex	Aperçu	Utilisation
garage-50	#f5f5f4		Texte très clair
garage-100	#e7e5e4		Texte principal
garage-200	#d6d3d1		Texte casiers
garage-400	#78716c		Bordures casiers
garage-500	#57534e		Bordures inputs/cartes
garage-600	#44403c		Fond cartes
garage-700	#292524		Fond page (body)
garage-800	#1c1917		Fond modals
garage-950	#050403		Sidebar
accent	#f97316		Couleur accent (boutons, titres, nav active)

11.2 Classes CSS personnalisées

Classe	Description
.btn-primary	Bouton orange accent, texte blanc
.btn-secondary	Bouton gris foncé avec bordure
.btn-danger	Bouton rouge
.card	Carte avec fond garage-600, bordure, coins arrondis, ombre
.input-field	Champ de saisie sombre, focus ring accent
.select-field	Sélecteur sombre, focus ring accent

12. Composants React

L'application comprend **9 composants réutilisables** dans `src/components/` :

Composant	Props principales	Rôle
Sidebar	—	Navigation latérale avec détection de route active
Modal	isOpen, onClose, title, size	Fenêtre modale générique (sm/md/lg/xl), fermeture Escape
ImageZoom	src, alt	Miniature 64x64 avec zoom plein écran au clic
ItemForm	reference, onSave, onCancel, initial?, presetLocationId?	Formulaire création/édition item avec upload photo Base64
ItemListPage	reference, title	Liste filtrée + recherche + grille responsive
ItemDetailModal	itemId, onClose, onRefresh	Fiche détail : historique, prêts, actions Vendu/Vide
MovementLog	reference	Historique des mouvements + historique masqué + export CSV
CategoriesManager	—	CRUD catégories avec filtre par référence
GridPlan	—	Plan 2D interactif : génération, affichage, modification, suppression
BackupRestore	—	Sauvegarde JSON et restauration avec confirmation

13. Annexes

Annexe A — Commandes utiles

```
# === Démarrage ===
docker compose up -d --build      # Construire et lancer
docker compose down               # Arrêter
docker compose down -v           # Arrêter + supprimer les données

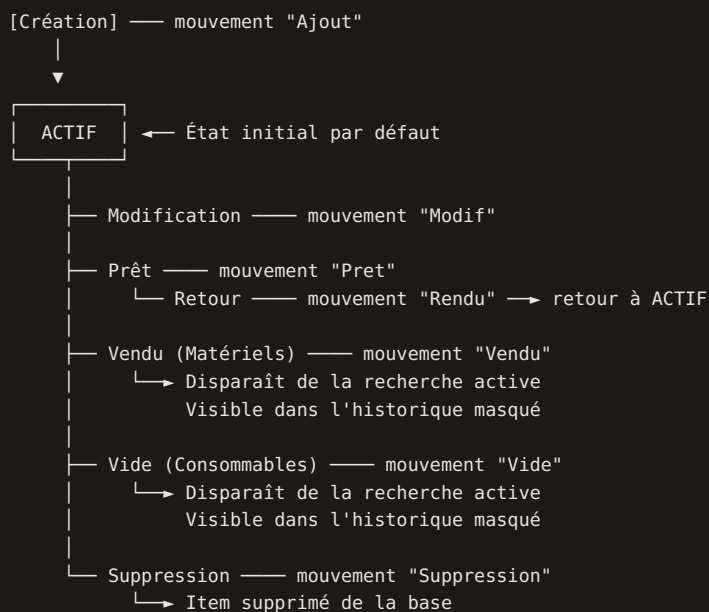
# === Logs ===
docker compose logs -f web        # Logs Next.js en temps réel
docker compose logs -f db        # Logs MySQL en temps réel

# === Base de données ===
docker exec toolmap-web ./node_modules/.bin/prisma db push  # Appliquer le schéma
docker exec toolmap-web ./node_modules/.bin/tsx prisma/seed.ts # Seed catégories
docker exec -it toolmap-db mysql -u toolmap -ptoolmap_pass_2024 toolmap # Shell MySQL

# === SSL ===
./generate-ssl.sh                # Régénérer les certificats

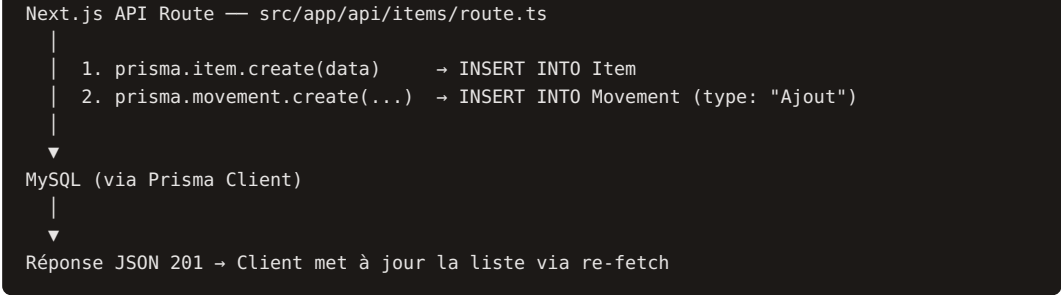
# === Maintenance ===
docker compose restart web        # Redémarrer Next.js
docker system prune -f           # Nettoyer les images/conteneurs inutilisés
```

Annexe B — Cycle de vie d'un Item



Annexe C — Flux de données (création d'item)

```
Navigateur (client)
|
| POST /api/items
| Body: { name, reference, photo, categoryId, locationId, ... }
|
▼
```



Annexe D — Format des photos

Les photos sont stockées dans le champ `photo` (LONGTEXT) sous forme de **Data URI Base64** :

```
data:image/jpeg;base64,/9j/4AAQSkZJRgABAQAA...
```

Limite	Valeur	Configuration
Taille upload Next.js	10 MB	<code>next.config.ts</code> → <code>serverActions.bodySizeLimit</code>
Taille paquet MySQL	64 MB	<code>docker-compose.yml</code> → <code>max_allowed_packet</code>

Annexe E — Sécurité

Mesure	Détail
HTTPS	Obligatoire, port 443, certificat SSL
Injection SQL	Protégé par Prisma ORM (requêtes paramétrées)
XSS	Protégé par React (escape automatique du JSX)
Mots de passe	Configurables via variables d'environnement
Upload	Limité à 10 MB par fichier
Authentification	Non implémentée (application locale/intranet)

Recommandation : Si l'application est exposée sur Internet, ajoutez une couche d'authentification (NextAuth.js ou similaire) et utilisez un certificat SSL valide (Let's Encrypt).

Annexe F — Dépannage

Problème	Cause probable	Solution
Page blanche / erreur 502	Next.js pas encore démarré	<code>docker compose logs web</code> — attendre « Ready »
Erreur connexion BDD	MySQL pas encore healthy	Attendre 30s, vérifier <code>docker compose logs db</code>
Certificat SSL refusé	Certificat auto-signé	Accepter l'exception dans le navigateur
Prisma schema error	BDD non initialisée	<code>docker exec toolmap-web ./node_modules/.bin/prisma db push</code>
Catégories manquantes	Seed non exécuté	<code>docker exec toolmap-web ./node_modules/.bin/tsx prisma/seed.ts</code>
Photo trop volumineuse	Dépasse 10 MB	Compresser l'image avant upload

Conteneur en restart loop	Erreur au démarrage	<code>docker compose logs web --tail 50</code> pour diagnostiquer
---------------------------	---------------------	---